

Mizar Evolution: Why First-Order Logic, Now

Connecting automatic proof, readable mathematics, and checked computation

TPP 2026, RIKEN AIP Tokyo Office, November 2026

presenter notes edition

Mizar Evo project

TPP 2026, RIKEN AIP Tokyo Office, November 16–17, 2026

Mizar Evolution: Why First-Order Logic, Now

Mizar Evolution: Why First-Order Logic, Now
Connecting automatic proof, readable mathematics, and checked computation
TPP 2026, RIKEN AIP Tokyo Office, November 2026
presenter notes edition

Mizar Evo project

TPP 2026, RIKEN AIP Tokyo Office, November 16–17, 2026

Read aloud:

- **Mizar is a proof checker. It has a fifty-year-old library, written in a readable mathematical language over first-order set theory.**
- **Mizar Evolution, or Mizar Evo, is a redesign of its language and tools. Today I will not list features. I will start from a puzzle about automatic provers, and use it to explain why we keep the logic first-order.**
- Everything I say has a label: fact, research hypothesis, or future direction.

0.2 - How To Read The Claims

deep dive

Level	Meaning	Marked how
fact	existing systems, published benchmarks, the Mizar Evo specification and main branch	no tag
research hypothesis	a claim that Mizar Evo is built to test	tagged in the text
future direction	a goal with no fixed date or design yet	tagged in the text

Code labels follow the Bialystok deck: exact MML excerpt, specification example, sketch.

- The specification and the implementation are different things. One frame near the end separates them.

2026-10-07

Mizar Evolution: Why First-Order Logic, Now

└ Part 0. Opening

└ 0.2 - How To Read The Claims

deep dive

0.2 - How To Read The Claims

deep dive

Level	Meaning	Marked how
fact	existing systems, published benchmarks, the Mizar Evo specification and main branch	no tag
research hypothesis	a claim that Mizar Evo is built to test	tagged in the text
future direction	a goal with no fixed date or design yet	tagged in the text

Code labels follow the Bialystok deck: exact MML excerpt, specification example, sketch.

- The specification and the implementation are different things. One frame near the end separates them.

Read aloud:

- Code labels follow the Bialystok deck: exact MML excerpt, specification example, sketch.
- The specification and the implementation are different things. One frame near the end separates them.

1.1 - Two Numbers That Look Alike

	Mizar 60 (ITP 2023)	Sledgehammer on the AFP (CICM 2015)
headline	58.4% proved	60.7% proved
counts	top-level theorems and lemmas of MML 1147 (57,897)	proof goals inside AFP developments (6,934)
premises	chosen from the whole library by learned methods	chosen from the development by the MePo filter
budget	portfolio limited to 420 CPU seconds	30 seconds per prover, four provers
status	ATP proof found, hammering mode	union trusted as oracle; about 50% with one-line replay

- Two hammers, two headline numbers, and they look almost the same.
- Do not compare them as one benchmark. The units, the premises, and the time budgets are all different.

Mizar Evolution: Why First-Order Logic, Now

└ Part 1. Two Hammers, Two Numbers

└ 1.1 - Two Numbers That Look Alike

1.1 - Two Numbers That Look Alike

	Mizar 60 (ITP 2023)	Sledgehammer on the AFP (CICM 2015)
headline	58.4% proved	60.7% proved
counts	top-level theorems and lemmas of MML 1147 (57,897)	proof goals inside AFP developments (6,934)
premises	chosen from the whole library by learned methods	chosen from the developments by the MePo filter
budget	portfolio limited to 420 CPU seconds	30 seconds per prover, four provers
status	ATP proof found, hammering mode	union trusted as oracle; about 50% with one-line replay

- Two hammers, two headline numbers, and they look almost the same.
- Do not compare them as one benchmark. The units, the premises, and the time budgets are all different.

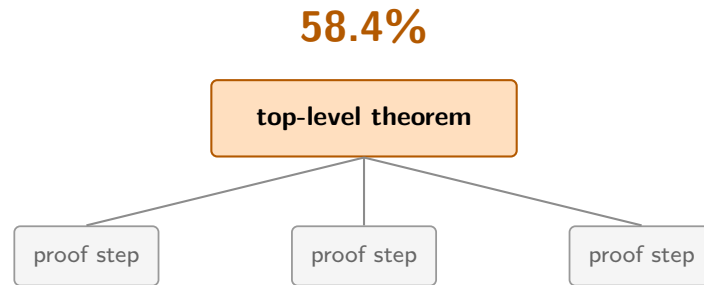
Read aloud:

- **Two hammers, two headline numbers, and they look almost the same.**
- **Do not compare them as one benchmark. The units, the premises, and the time budgets are all different.**

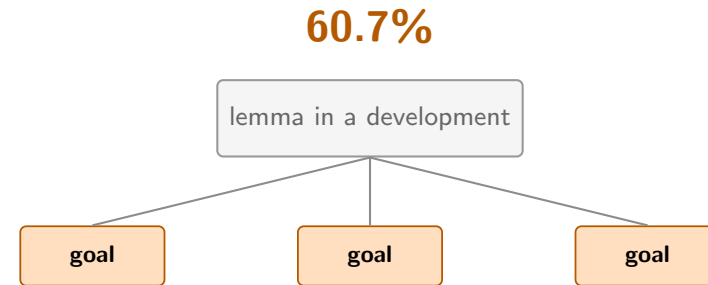
Optional notes and sources:

- A "hammer" is a tool that sends a goal to automatic provers. "Premises" are the facts the prover may use.
- Source: Jakubův et al. 2023, results 1-3; Blanchette et al. 2015, Section "Proof Automation with Sledgehammer", Figure 13.
- MizAR's 75% figure uses premises chosen from the library by a human or a machine. Keep it for questions.

1.2 - What Each Number Counts



Mizar 60 (ITP 2023) counts *whole top-level theorems and lemmas* 57,897 in MML 1147; premises from the whole library; 420 CPU s portfolio



Sledgehammer on the AFP (CICM 2015) counts *proof goals as they arise* 6,934 goals from 128 theories; 30 s per prover; union trusted as oracle

Same-looking percentages. Different units, different premise regimes, different budgets.

- MizAR asks: can the machine prove this whole theorem from the library, with no help from the author?
- The AFP study asks: can the machine close this goal where it appears, often inside a proof that a person has already written?
- Both are fair questions. They are not the same question.

Mizar Evolution: Why First-Order Logic, Now

└ Part 1. Two Hammers, Two Numbers

└ 1.2 - What Each Number Counts

1.2 - What Each Number Counts



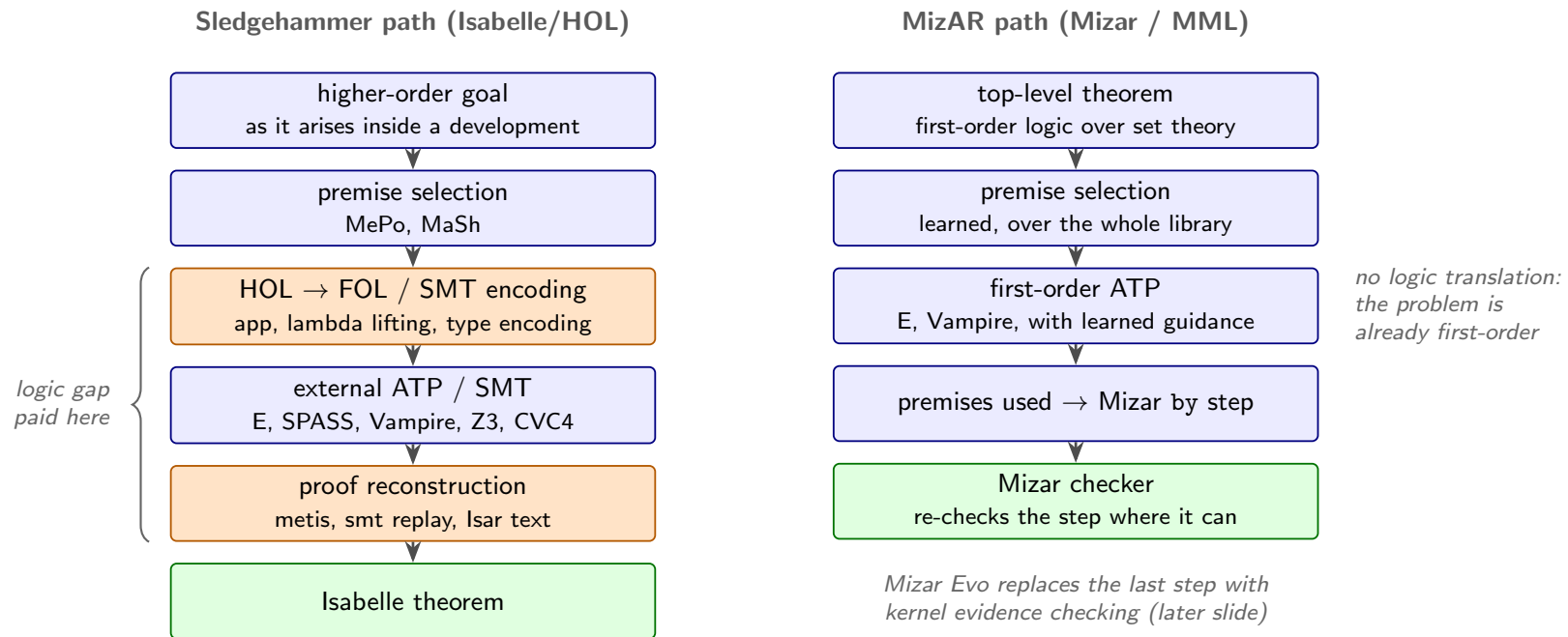
Same-looking percentages. Different units, different premise regimes, different budgets.

- MizAR asks: can the machine prove this whole theorem from the library, with no help from the author?
- The AFP study asks: can the machine close this goal where it appears, often inside a proof that a person has already written?
- Both are fair questions. They are not the same question.

Read aloud:

- **MizAR asks: can the machine prove this whole theorem from the library, with no help from the author?**
- **The AFP study asks: can the machine close this goal where it appears, often inside a proof that a person has already written?**
- Both are fair questions. They are not the same question.

1.3 - Two Paths To A First-Order Prover



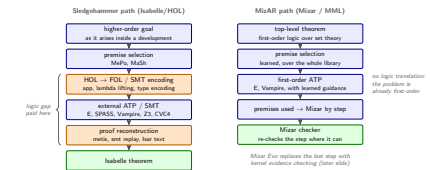
- Sledgehammer translates a higher-order goal into first-order or SMT problems. Then it rebuilds the proof inside Isabelle.
- MizAR gives the prover a problem that is already first-order. The gap between the checker and the ATP problem is short.

Mizar Evolution: Why First-Order Logic, Now

└ Part 1. Two Hammers, Two Numbers

└ 1.3 - Two Paths To A First-Order Prover

1.3 - Two Paths To A First-Order Prover



- Sledgehammer translates a higher-order goal into first-order or SMT problems. Then it rebuilds the proof inside Isabelle.
- MizAR gives the prover a problem that is already first-order. The gap between the checker and the ATP problem is short.

Read aloud:

- **Sledgehammer translates a higher-order goal into first-order or SMT problems. Then it rebuilds the proof inside Isabelle.**
- **MizAR gives the prover a problem that is already first-order. The gap between the checker and the ATP problem is short.**

Optional notes and sources:

- Both paths describe existing systems (Blanchette, Kaliszyk, Paulson, Urban 2016; Jakubův et al. 2023). The orange boxes are the translation and reconstruction layers.
- MizAR's headline percentages count ATP proofs. The Mizar checker re-checks the resulting by step when the inference is within its strength.

1.4 - The Question

A result of how we measure, or a result of the design?

- **Research hypothesis:** in a first-order library, the distance between the checker's own logic and the ATP problem is short. A higher-order system must cross that distance with translation and reconstruction, and that has a cost.
- **Fact:** Sledgehammer showed that higher-order systems can use first-order ATPs very well.
- **Research hypothesis:** we do not yet know if "higher-order plus translation" is the best design for using ATPs. There is almost no modern first-order ITP to compare with.
- **Mizar Evo is built to test this hypothesis with a modern verifier, library, and provers.**

Mizar Evolution: Why First-Order Logic, Now

└ Part 1. Two Hammers, Two Numbers

└ 1.4 - The Question

1.4 - The Question

A result of how we measure, or a result of the design?

- Research hypothesis: in a first-order library, the distance between the checker's own logic and the ATP problem is short. A higher-order system must cross that distance with translation and reconstruction, and that has a cost.
- Fact: Sledgehammer showed that higher-order systems can use first-order ATPs very well.
- Research hypothesis: we do not yet know if "higher-order plus translation" is the best design for using ATPs. There is almost no modern first-order ITP to compare with.
- Mizar Evo is built to test this hypothesis with a modern verifier, library, and provers.

Read aloud:

- **A result of how we measure, or a result of the design?**
- **Research hypothesis: in a first-order library, the distance between the checker's own logic and the ATP problem is short. A higher-order system must cross that distance with translation and reconstruction, and that has a cost.**
- Fact: Sledgehammer showed that higher-order systems can use first-order ATPs very well.
- Research hypothesis: we do not yet know if "higher-order plus translation" is the best design for using ATPs. There is almost no modern first-order ITP to compare with.
- **Mizar Evo is built to test this hypothesis with a modern verifier, library, and provers.**

2.1 - Functions In Higher-Order Logic

Schematic HOL statement **sketch** (Isabelle/HOL-style notation)

```
f :: 'a => 'b      x :: 'a
```

```
P (f x)
```

- In HOL, function types and function application are part of the logic itself.
- Functions as arguments, functions that return functions, partial application, lambda: all can be written directly.
- This is very convenient for the person who writes mathematics.
- But E and Vampire are first-order provers. The higher-order structure must be encoded before they can see it.

Mizar Evolution: Why First-Order Logic, Now

└ Part 2. Where Complexity Lives

└ 2.1 - Functions In Higher-Order Logic

2.1 - Functions In Higher-Order Logic

Schematic HOL statement **sketch** (Isabelle/HOL-style notation)

$f :: 'a \rightarrow 'b$ $x :: 'a$

$P (f x)$

- In HOL, function types and function application are part of the logic itself.
- Functions as arguments, functions that return functions, partial application, lambda: all can be written directly.
- This is very convenient for the person who writes mathematics.
- But E and Vampire are first-order provers. The higher-order structure must be encoded before they can see it.

Read aloud:

- Schematic HOL statement.
- **In HOL, function types and function application are part of the logic itself.**
- Functions as arguments, functions that return functions, partial application, lambda: all can be written directly.
- **This is very convenient for the person who writes mathematics.**
- **But E and Vampire are first-order provers. The higher-order structure must be encoded before they can see it.**

2.2 - The Cost Appears At The ATP Boundary

Encoding steps, schematic **sketch**

<code>F X</code>	<code>-> app(F, X)</code>	
<code>(%x. t) ...</code>	<code>-> fresh constant + defining axioms</code>	<code>(lambda lifting)</code>
<code>polymorphic types</code>	<code>-> type guards or type tags</code>	
<code>Boolean-valued terms</code>	<code>-> extra encoding</code>	

- Applying a variable function becomes an explicit `app` symbol. Lambdas are lifted out or turned into combinators. Types become guards or tags.
- After the prover succeeds, the proof must come back into Isabelle: a `metis` call, an `smt` replay, or generated `Isar` text. This step is called reconstruction.
- This is a great piece of engineering. It is also a layer that a first-order system does not need.
- HOL gets the convenience first and pays at the ATP boundary.

Mizar Evolution: Why First-Order Logic, Now

└ Part 2. Where Complexity Lives

└ 2.2 - The Cost Appears At The ATP Boundary

2.2 - The Cost Appears At The ATP Boundary

Encoding steps, schematic sketch

$\lambda x. t$	$\rightarrow$	<code>app(P, X)</code>
$(\lambda x. t) \dots$	$\rightarrow$	fresh constant + defining axioms (lambda lifting)
polymorphic types	$\rightarrow$	type guards or type tags
Boolean-valued terms	$\rightarrow$	extra encoding

- Applying a variable function becomes an explicit app symbol. Lambdas are lifted out or turned into combinators. Types become guards or tags.
- After the prover succeeds, the proof must come back into Isabelle: a `metis` call, an `smt` replay, or generated `Isar` text. This step is called reconstruction.
- This is a great piece of engineering. It is also a layer that a first-order system does not need.
- HOL gets the convenience first and pays at the ATP boundary.

Read aloud:

- Encoding steps, schematic.
- **Applying a variable function becomes an explicit app symbol. Lambdas are lifted out or turned into combinators. Types become guards or tags.**
- **After the prover succeeds, the proof must come back into Isabelle: a `metis` call, an `smt` replay, or generated `Isar` text. This step is called reconstruction.**
- This is a great piece of engineering. It is also a layer that a first-order system does not need.
- HOL gets the convenience first and pays at the ATP boundary.

Optional notes and sources:

- Source: Meng and Paulson 2008 (translating higher-order clauses to first-order clauses); Blanchette, Böhme, Popescu, Smallbone 2016 (type encodings); Blanchette, Kaliszyk, Paulson, Urban 2016 (survey); Schurr, Fleury, Desharnais 2021 (reconstruction).
- The exact encodings that Sledgehammer uses depend on the prover and the options. This frame is schematic.

2.3 - Functions In Set Theory: Mizar's Foundation

Function application is a defined set-theoretic relation **exact MML excerpt**

```
func f.x -> set means  
:Def2:  
[x,it] in f if x in dom f otherwise it = {};
```

A typed function is a soft type over partial functions **exact MML excerpt**

```
definition  
  let X,Y;  
  mode Function of X,Y is quasi_total PartFunc of X,Y;  
end;
```

- Here a function is an ordinary first-order object: a set of pairs. Application is defined, not built into the logic.
- Nothing in the base logic has to be higher-order.
- The price: if you write raw set theory, the domain, the graph, functionality, and membership all appear in the text.

Mizar Evolution: Why First-Order Logic, Now

└ Part 2. Where Complexity Lives

└ 2.3 - Functions In Set Theory: Mizar's Foundation

2.3 - Functions In Set Theory: Mizar's Foundation

Function application is a defined set-theoretic relation exact MML excerpt

```
func f.x -> set means
:Def2:
[x,it] in f if x in dom f otherwise it = {};
```

A typed function is a soft type over partial functions exact MML excerpt

```
definition
let X,Y;
mode Function of X,Y is quasi_total PartFunc of X,Y;
end;
```

- Here a function is an ordinary first-order object: a set of pairs. Application is defined, not built into the logic.
- Nothing in the base logic has to be higher-order.
- The price: if you write raw set theory, the domain, the graph, functionality, and membership all appear in the text.

Read aloud:

- Function application is a defined set-theoretic relation.
- A typed function is a soft type over partial functions.
- **Here a function is an ordinary first-order object: a set of pairs. Application is defined, not built into the logic.**
- **Nothing in the base logic has to be higher-order.**
- The price: if you write raw set theory, the domain, the graph, functionality, and membership all appear in the text.

Optional notes and sources:

- Source: current MML `funct_1.miz`, lines 138-140 (`FUNCT_1: def 2`), and `funct_2.miz`, lines 87-90 (checked October 7, 2026). URLs:
https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz,
https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz. GPL-3.0-or-later / CC-BY-SA-3.0-or-later.
- `quasi_total` (`FUNCT_2`, lines 36-44) says the domain is all of X unless Y is empty.

2.4 - Mizar's Fifty-Year Answer: Hide The Details In The Language

What the author actually writes **sketch** (valid in current Mizar and Mizar Evo)

```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- It is the same set-theoretic function. But the author writes Function of X,Y and f.x, not pairs and domains.
- Soft types, modes, attributes, registrations, schemes, and declarative proofs are not just a nicer way to write the same thing. They are the language design that lifts first-order set theory up to readable mathematics.
- Mizar has walked in this direction for fifty years. Mizar Evo keeps walking.

Mizar Evolution: Why First-Order Logic, Now

└ Part 2. Where Complexity Lives

└ 2.4 - Mizar's Fifty-Year Answer: Hide The Details In The Language

2.4 - Mizar's Fifty-Year Answer: Hide The Details In The Language

What the author actually writes **sketch** (valid in current Mizar and Mizar Evo)

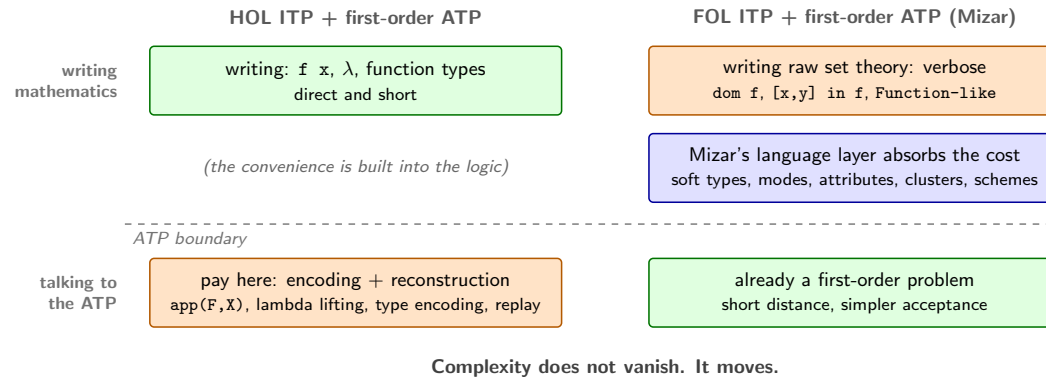
```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- It is the same set-theoretic function. But the author writes Function of X,Y and f.x, not pairs and domains.
- Soft types, modes, attributes, registrations, schemes, and declarative proofs are not just a nicer way to write the same thing. They are the language design that lifts first-order set theory up to readable mathematics.
- Mizar has walked in this direction for fifty years. Mizar Evo keeps walking.

Read aloud:

- What the author actually writes.
- **It is the same set-theoretic function. But the author writes Function of X,Y and f.x, not pairs and domains.**
- **Soft types, modes, attributes, registrations, schemes, and declarative proofs are not just a nicer way to write the same thing. They are the language design that lifts first-order set theory up to readable mathematics.**
- **Mizar has walked in this direction for fifty years. Mizar Evo keeps walking.**

2.5 - Where You Pay



*HOL and FOL do not remove complexity.
They put it in different places.*

- HOL pays at the ATP boundary. FOL pays at the keyboard, and Mizar's language takes on that payment.
- Mizar Evo's choice: keep the base logic first-order, and hide the human-facing complexity in the language.

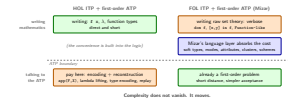
2026-10-07

Mizar Evolution: Why First-Order Logic, Now

└ Part 2. Where Complexity Lives

└ 2.5 - Where You Pay

2.5 - Where You Pay



*HOL and FOL do not remove complexity.
They put it in different places.*

- HOL pays at the ATP boundary. FOL pays at the keyboard, and Mizar's language takes on that payment.
- Mizar Evo's choice: keep the base logic first-order, and hide the human-facing complexity in the language.

Read aloud:

- **HOL and FOL do not remove complexity. They put it in different places.**
- **HOL pays at the ATP boundary. FOL pays at the keyboard, and Mizar's language takes on that payment.**
- **Mizar Evo's choice: keep the base logic first-order, and hide the human-facing complexity in the language.**

2.6 - The Trade-Off In One Table

deep dive

	HOL ITP + ATP	FOL ITP + ATP
writing	higher-order features, short text	long text if written raw
functions	basic higher-order objects	first-order set-theoretic objects
ATP connection	encoding needed	short distance
reconstruction	must cross the logic gap back	simpler in principle
the language must provide	HOL abstraction	abstraction that hides first-order details

- This table is an interpretation, not a measurement. The measurement is what Mizar Evo must produce in 2027.

2026-10-07

Mizar Evolution: Why First-Order Logic, Now

└ Part 2. Where Complexity Lives

└ 2.6 - The Trade-Off In One Table

deep dive

2.6 - The Trade-Off In One Table

	HOL ITP + ATP	FOL ITP + ATP
writing	higher-order features, short text	long text if written as
functions	basic higher-order objects	first-order set-theoretic objects
ATP connection	encoding needed	short distance
reconstruction	must cross the logic gap back	simpler in principle
the language must provide	HOL abstraction	abstraction that hides first-order details

- This table is an interpretation, not a measurement. The measurement is what Mizar Evo must produce in 2027.

Read aloud:

- This table is an interpretation, not a measurement. The measurement is what Mizar Evo must produce in 2027.

3.1 - Keep The Logic, Modernize The Language

*Keep the mathematical layer.
Rebuild the tools under it and around it.*

- Mizar Evo keeps first-order logic and Tarski-Grothendieck set theory as the base logic.
- It keeps soft types, modes, attributes, registrations, structures, and declarative proofs.
- It makes hidden choices explicit, unifies the generic mechanisms, makes automation traceable, and puts everything on a modern compiler architecture.
- The next frames show what "modernize the language" means: templates, algorithms, the trust boundary, and the infrastructure.

Mizar Evolution: Why First-Order Logic, Now

└ Part 3. Modernizing Mizar's Answer

└ 3.1 - Keep The Logic, Modernize The Language

3.1 - Keep The Logic, Modernize The Language

*Keep the mathematical layer.
Rebuild the tools under it and around it.*

- Mizar Evo keeps first-order logic and Tarski-Grothendieck set theory as the base logic.
- It keeps soft types, modes, attributes, registrations, structures, and declarative proofs.
- It makes hidden choices explicit, unifies the generic mechanisms, makes automation traceable, and puts everything on a modern compiler architecture.
- The next frames show what "modernize the language" means: templates, algorithms, the trust boundary, and the infrastructure.

Read aloud:

- **Keep the mathematical layer. Rebuild the tools under it and around it.**
- **Mizar Evo keeps first-order logic and Tarski-Grothendieck set theory as the base logic.**
- It keeps soft types, modes, attributes, registrations, structures, and declarative proofs.
- **It makes hidden choices explicit, unifies the generic mechanisms, makes automation traceable, and puts everything on a modern compiler architecture.**
- The next frames show what "modernize the language" means: templates, algorithms, the trust boundary, and the infrastructure.

3.2 - Templates: Generic Mathematics Without Higher-Order Logic

A template is a definition block with parameters **specification example**

```
definition
  let T be type;
  struct MagmaStr[T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;
```

- Parameters can be types, values, predicates, or functors. Classical Mizar schemes become theorems with a predicate parameter, in the same system.
- Templates add no unrestricted second-order quantification to the logic. Each instantiation is checked and produces first-order obligations.
- Generic mathematics lives in the surface language, not in the base logic.

Mizar Evolution: Why First-Order Logic, Now

└ Part 3. Modernizing Mizar's Answer

└ 3.2 - Templates: Generic Mathematics Without Higher-Order Logic

3.2 - Templates: Generic Mathematics Without Higher-Order Logic

A template is a definition block with parameters **specification example**

```
definition
  let T be type;
  struct MagmaStr(T) where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;
```

- Parameters can be types, values, predicates, or functors. Classical Mizar schemes become theorems with a predicate parameter, in the same system.
- Templates add no unrestricted second-order quantification to the logic. Each instantiation is checked and produces first-order obligations.
- Generic mathematics lives in the surface language, not in the base logic.

Read aloud:

- **A template is a definition block with parameters.**
- **Parameters can be types, values, predicates, or functors. Classical Mizar schemes become theorems with a predicate parameter, in the same system.**
- **Templates add no unrestricted second-order quantification to the logic. Each instantiation is checked and produces first-order obligations.**
- Generic mathematics lives in the surface language, not in the base logic.

Optional notes and sources:

- Source: `doc/spec/en/18.templates.md`, sections 18.1-18.2 and 18.8.

Induction as a theorem with a predicate parameter specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- A predicate parameter gives a family of first-order theorems, one for each predicate you put in.
- Instantiation is explicit: by `NatInduction[P]`, `Base`, `Step` after a `defpred`.
- Functor parameters are schema-level symbols, not sets. This keeps the logic first-order.
- Details: Bialystok deck, Story 6 (`PermProduct [T]`, qua views), and Backup 4 here.

Mizar Evolution: Why First-Order Logic, Now

└ Part 3. Modernizing Mizar's Answer

└ 3.3 - Schemes Become Ordinary Templates

deep dive

Induction as a theorem with a predicate parameter **specification example**

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- A predicate parameter gives a family of first-order theorems, one for each predicate you put in.
- Instantiation is explicit: by `NatInduction[P]`, `Base`, `Step` after a `defpred`.
- Functor parameters are schema-level symbols, not sets. This keeps the logic first-order.
- Details: Bialystok deck, Story 6 (`PermProduct [T]`, `qua views`), and Backup 4 [here](#).

Read aloud:

- Induction as a theorem with a predicate parameter.
- A predicate parameter gives a family of first-order theorems, one for each predicate you put in.
- Instantiation is explicit: by `NatInduction[P]`, `Base`, `Step` after a `defpred`.
- Functor parameters are schema-level symbols, not sets. This keeps the logic first-order.
- Details: Bialystok deck, Story 6 (`PermProduct [T]`, `qua views`), and Backup 4 [here](#).

3.4 - Algorithms Are A Second Pillar

- **Algorithms in Mizar Evo are not a replacement for higher-order functions. They answer a different need: reasoning about algorithms, and computation you can check.**
- A historical tendency, not a necessity: first-order logic has complete calculi, and a culture of automatic search grew around them, from resolution to superposition and saturation.
- The LCF and HOL family grew a culture of programmable proof construction: tactics and tacticals.
- **Mizar used declarative proofs and the automation built into the system. It never had a tactic language that users could program.**
- Mizar Evo's algorithm is not a tactic language added on top. It is a procedure with a contract, and the system verifies it.

Mizar Evolution: Why First-Order Logic, Now

└ Part 3. Modernizing Mizar's Answer

└ 3.4 - Algorithms Are A Second Pillar

3.4 - Algorithms Are A Second Pillar

- Algorithms in Mizar Evo are not a replacement for higher-order functions. They answer a different need: reasoning about algorithms, and computation you can check.
- A historical tendency, not a necessity: first-order logic has complete calculi, and a culture of automatic search grew around them, from resolution to superposition and saturation.
- The LCF and HOL family grew a culture of programmable proof construction: tactics and tacticals.
- Mizar used declarative proofs and the automation built into the system. It never had a tactic language that users could program.
- Mizar Evo's algorithm is not a tactic language added on top. It is a procedure with a contract, and the system verifies it.

Read aloud:

- **Algorithms in Mizar Evo are not a replacement for higher-order functions. They answer a different need: reasoning about algorithms, and computation you can check.**
- A historical tendency, not a necessity: first-order logic has complete calculi, and a culture of automatic search grew around them, from resolution to superposition and saturation.
- The LCF and HOL family grew a culture of programmable proof construction: tactics and tacticals.
- **Mizar used declarative proofs and the automation built into the system. It never had a tactic language that users could program.**
- Mizar Evo's algorithm is not a tactic language added on top. It is a procedure with a contract, and the system verifies it.

3.5 - Algorithms: Contracts, Proofs, Computation

Euclid's algorithm with a contract **specification example**

```
terminating algorithm euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a;  var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);  decreasing y;
    const r := x mod y;  x := y;  y := r;
  end;
  return x;
end;
```

- Contracts, invariants, and termination measures become first-order proof obligations, checked like theorems.
- Two roles: a verified automation procedure, and a checked computation run by by computation. Status: specified. MVM execution and code extraction come later.

Mizar Evolution: Why First-Order Logic, Now

└ Part 3. Modernizing Mizar's Answer

└ 3.5 - Algorithms: Contracts, Proofs, Computation

3.5 - Algorithms: Contracts, Proofs, Computation

Euclid's algorithm with a contract specification example

```

terminating algorithm euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a; var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y); decreasing y;
    const r := x mod y; x := y; y := r;
  end
  return x;
end;

```

- Contracts, invariants, and termination measures become first-order proof obligations, checked like theorems.
- Two roles: a verified automation procedure, and a checked computation run by by computation. Status: specified. MVM execution and code extraction come later.

Read aloud:

- **Euclid's algorithm with a contract.**
- **Contracts, invariants, and termination measures become first-order proof obligations, checked like theorems.**
- **Two roles: a verified automation procedure, and a checked computation run by by computation. Status: specified. MVM execution and code extraction come later.**

Optional notes and sources:

- Source: doc/spec/en/20.algorithm_and_verification.md, section 20.12 (condensed: the enclosing definition block and let a, b be Nat; are omitted).

Future direction, not current capability:

- number-theoretic and combinatorial algorithms; symbolic algorithms; optimization procedures;
- later: cryptographic algorithms and protocols; quantum and hybrid classical-quantum algorithms.

One workflow for all of them:

- 1 write the procedure; 2. state the contract; 3. give invariants and termination; 4. generate proof obligations; 5. prove them with ATPs and check them in the kernel; 6. run on concrete inputs; 7. extract code later.
- **Algorithms make the gap between tactics and application algorithms smaller. The targets above are future directions.**

Mizar Evolution: Why First-Order Logic, Now

└ Part 3. Modernizing Mizar's Answer

└ 3.6 - Future Targets For Algorithms

deep dive

3.6 - Future Targets For Algorithms

deep dive

Future direction, not current capability:

- number-theoretic and combinatorial algorithms; symbolic algorithms; optimization procedures;
- later: cryptographic algorithms and protocols; quantum and hybrid classical-quantum algorithms.

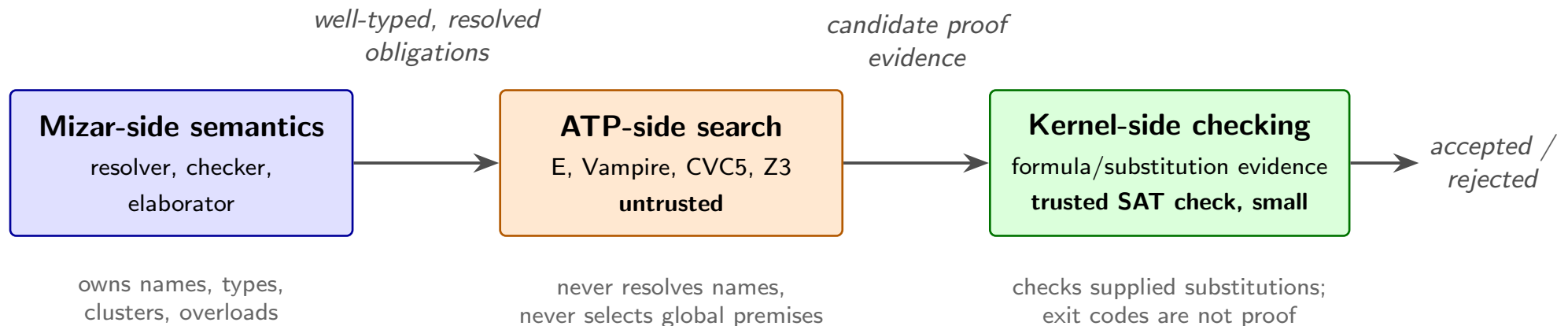
One workflow for all of them:

- write the procedure; 2. state the contract; 3. give invariants and termination; 4. generate proof obligations; 5. prove them with ATPs and check them in the kernel; 6. run on concrete inputs; 7. extract code later.
- **Algorithms make the gap between tactics and application algorithms smaller. The targets above are future directions.**

Read aloud:

- number-theoretic and combinatorial algorithms; symbolic algorithms; optimization procedures;
- later: cryptographic algorithms and protocols; quantum and hybrid classical-quantum algorithms.
- write the procedure; 2. state the contract; 3. give invariants and termination; 4. generate proof obligations; 5. prove them with ATPs and check them in the kernel; 6. run on concrete inputs; 7. extract code later.
- **Algorithms make the gap between tactics and application algorithms smaller. The targets above are future directions.**

3.7 - Search Outside, Trust Inside



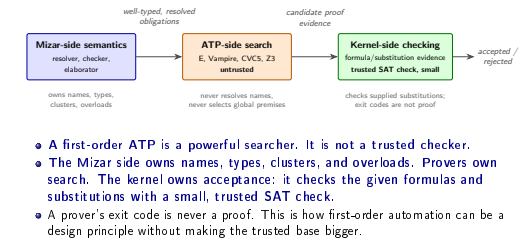
- A first-order ATP is a powerful searcher. It is not a trusted checker.
- The Mizar side owns names, types, clusters, and overloads. Provers own search. The kernel owns acceptance: it checks the given formulas and substitutions with a small, trusted SAT check.
- A prover's exit code is never a proof. This is how first-order automation can be a design principle without making the trusted base bigger.

Mizar Evolution: Why First-Order Logic, Now

└ Part 3. Modernizing Mizar's Answer

└ 3.7 - Search Outside, Trust Inside

3.7 - Search Outside, Trust Inside



Read aloud:

- **A first-order ATP is a powerful searcher. It is not a trusted checker.**
- **The Mizar side owns names, types, clusters, and overloads. Provers own search. The kernel owns acceptance: it checks the given formulas and substitutions with a small, trusted SAT check.**
- A prover's exit code is never a proof. This is how first-order automation can be a design principle without making the trusted base bigger.

Optional notes and sources:

- Source: `doc/design/architecture/en/08.reasoning_boundary.md`; Bialystok deck, Story 4; Backup 5 here shows what the evidence contains.

3.8 - And Mizar Itself Is Fifty Years Old

Seen after fifty years of MML	Mizar Evo
the article as the unit of dependency	modules with explicit imports
global name management	namespaces, fully qualified names
weak distribution and versioning	packages, SemVer, lock files
full rebuilds	incremental builds with dependency fingerprints
ATP as an add-on, automation hard to see	first-class ATP pipeline, resolution traces, kernel evidence
weak IDE and machine-readable I/O	LSP, structured diagnostics, agent interface

- **Keeping Mizar's mathematical ideas and keeping 1970s software architecture are two different things.**
- **Of course, we also modernize the whole development infrastructure. The Bialystok deck has the details. Here one table is enough.**

Mizar Evolution: Why First-Order Logic, Now

└ Part 3. Modernizing Mizar's Answer

└ 3.8 - And Mizar Itself Is Fifty Years Old

3.8 - And Mizar Itself Is Fifty Years Old

Seen after fifty years of MML	Mizar Evo
the article as the unit of dependency	modules with explicit imports
global name management	namespaces, fully qualified names
weak distribution and versioning	packages, SemVer, lock files
full rebuilds	incremental builds with dependency fingerprints
ATP as an add-on, automation hard to see	first-class ATP pipeline, resolution traces, kernel evidence
weak IDE and machine-readable I/O	LSP, structured diagnostics, agent interface

- Keeping Mizar's mathematical ideas and keeping 1970s software architecture are two different things.
- Of course, we also modernize the whole development infrastructure. The Bialystok deck has the details. Here one table is enough.

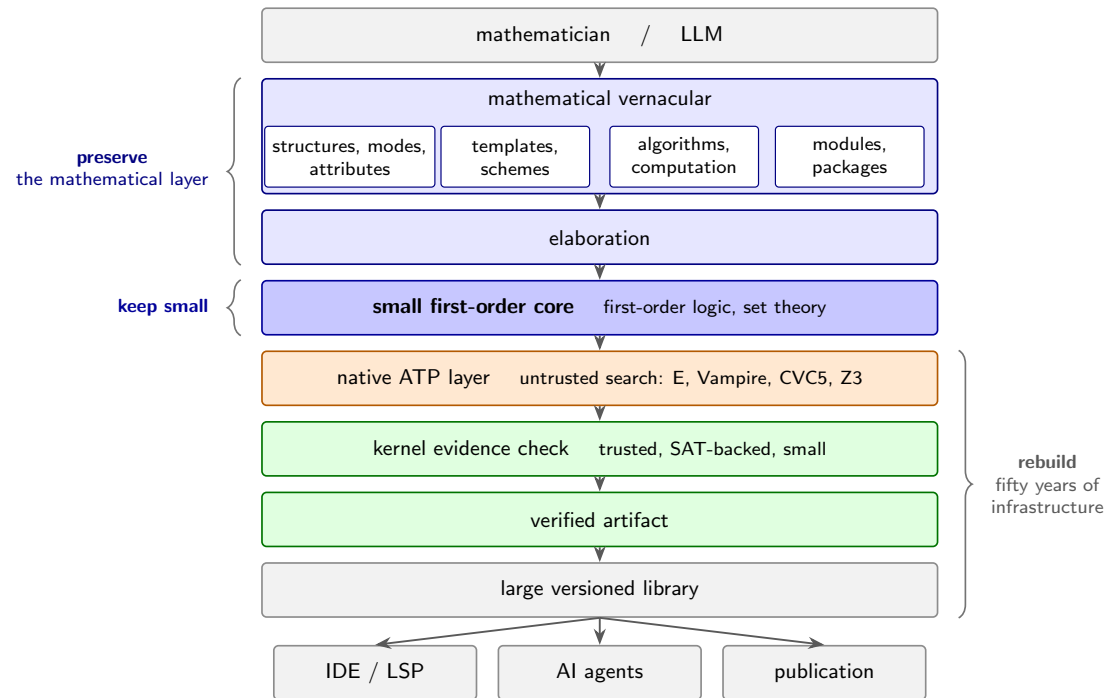
Read aloud:

- **Keeping Mizar's mathematical ideas and keeping 1970s software architecture are two different things.**
- **Of course, we also modernize the whole development infrastructure. The Bialystok deck has the details. Here one table is enough.**

Optional notes and sources:

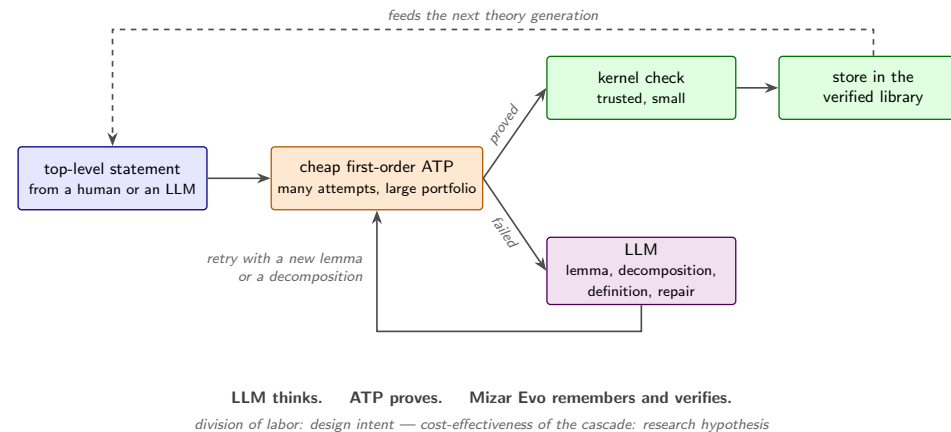
- This is not a criticism. It is software engineering that was not common fifty years ago, brought into a formal mathematics environment.
- Details: Bialystok deck, Stories 1, 3, 5, 8; Backups 6-8 here.

4.1 - The Whole Picture



*Rich mathematics above. A small first-order logic below.
Modern infrastructure around.*

4.2 - LLM Thinks, ATP Proves, Mizar Evo Remembers And Verifies



- **LLMs:** theories, definitions, strategy, lemmas, recovery from failure. **ATPs:** cheap, repeated first-order search. **Mizar Evo:** representation, verified library, trusted checking, origin of each fact.
- Research hypothesis: when machines generate a lot of mathematics, cheap ATP checking becomes more valuable. The cost and benefit of this loop are not yet measured.
- Current specification: ATPs receive only cited premises and local hypotheses. A library-wide hammer is research for 2027.

Mizar Evolution: Why First-Order Logic, Now

└ Part 4. The AI Era

└ 4.2 - LLM Thinks, ATP Proves, Mizar Evo Remembers And Verifies

4.2 - LLM Thinks, ATP Proves, Mizar Evo Remembers And Verifies



- LLMs: theories, definitions, strategy, lemmas, recovery from failure. ATPs: cheap, repeated first-order search. Mizar Evo: representation, verified library, trusted checking, origin of each fact.
- Research hypothesis: when machines generate a lot of mathematics, cheap ATP checking becomes more valuable. The cost and benefit of this loop are not yet measured.
- Current specification: ATPs receive only cited premises and local hypotheses. A library-wide hammer is research for 2027.

Read aloud:

- **LLMs: theories, definitions, strategy, lemmas, recovery from failure. ATPs: cheap, repeated first-order search. Mizar Evo: representation, verified library, trusted checking, origin of each fact.**
- Research hypothesis: when machines generate a lot of mathematics, cheap ATP checking becomes more valuable. The cost and benefit of this loop are not yet measured.
- Current specification: ATPs receive only cited premises and local hypotheses. A library-wide hammer is research for 2027.

Optional notes and sources:

- Source: `doc/spec/en/21.source_code_annotation_and_atp.md`, section 21.7.2, item 4 (no automatic premise selection from the global library);
`doc/design/architecture/en/21.ai_agent_interface.md` (edit classes).

4.3 - Where The Project Stands (October 2026)

- **Specification:** 24 chapters plus appendices. English is the canonical language.
- **Implemented on the main branch:** the Rust frontend (lexer, parser, syntax tree); name resolution and type checking on an alpha corpus; proof-obligation generation with deterministic discharge; ATP problem encoding and candidate evidence; SAT-backed kernel evidence checking; cache, fingerprint, and build-scheduling milestones.
- **In progress:** end-to-end integration from source to verified artifact; the LSP server; the documentation generator.
- **Planned for later:** MVM execution, code extraction, library-wide premise selection, MML migration.
- This talk does not claim any end-to-end results with external provers.

Mizar Evolution: Why First-Order Logic, Now

└ Part 4. The AI Era

└ 4.3 - Where The Project Stands (October 2026)

4.3 - Where The Project Stands (October 2026)

- Specification: 24 chapters plus appendices. English is the canonical language.
- Implemented on the main branch: the Rust frontend (lexer, parser, syntax tree); name resolution and type checking on an alpha corpus; proof-obligation generation with deterministic discharge; ATP problem encoding and candidate evidence; SAT-backed kernel evidence checking; cache, fingerprint, and build-scheduling milestones.
- In progress: end-to-end integration from source to verified artifact; the LSP server; the documentation generator.
- Planned for later: MVM execution, code extraction, library-wide premise selection, MML migration.
- This talk does not claim any end-to-end results with external provers.

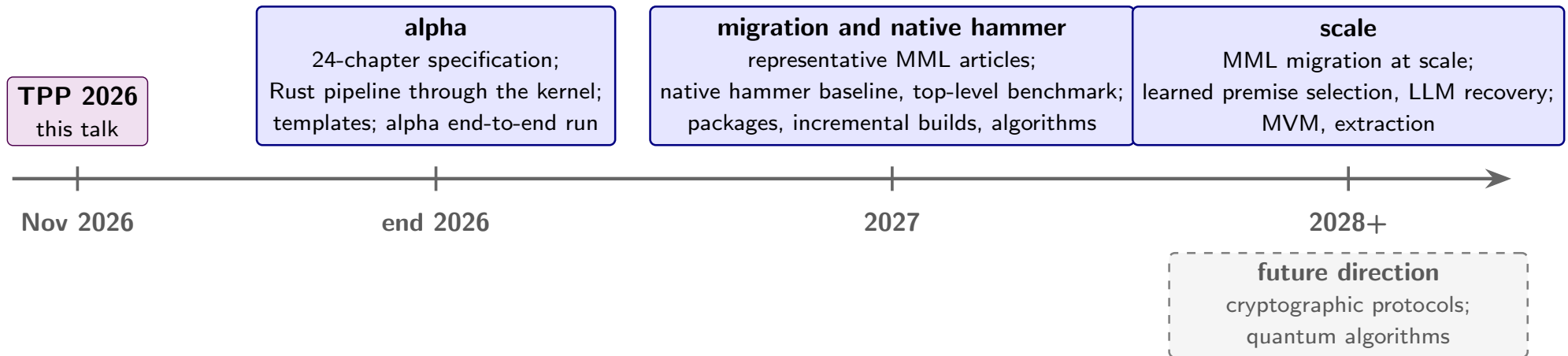
Read aloud:

- **Specification: 24 chapters plus appendices. English is the canonical language.**
- **Implemented on the main branch: the Rust frontend (lexer, parser, syntax tree); name resolution and type checking on an alpha corpus; proof-obligation generation with deterministic discharge; ATP problem encoding and candidate evidence; SAT-backed kernel evidence checking; cache, fingerprint, and build-scheduling milestones.**
- **In progress: end-to-end integration from source to verified artifact; the LSP server; the documentation generator.**
- **Planned for later: MVM execution, code extraction, library-wide premise selection, MML migration.**
- This talk does not claim any end-to-end results with external provers.

Optional notes and sources:

- Source: `doc/design/todo.md`, Crate Status, as of October 7, 2026. Re-check before the talk.

5.1 - Roadmap



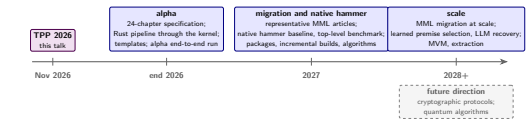
- 2026: finish the specification, the Rust pipeline through the kernel, template processing, and an alpha end-to-end run.
- 2027: migrate representative MML articles, build a native hammer baseline, and benchmark on top-level theorems, in the same units MizAR uses.
- 2028 and later: scale up the migration; learned premise selection; LLM-assisted failure recovery; MVM and extraction. Cryptographic and quantum targets stay future directions.

Mizar Evolution: Why First-Order Logic, Now

└ Part 5. Roadmap And Closing

└ 5.1 - Roadmap

5.1 - Roadmap



- 2026: finish the specification, the Rust pipeline through the kernel, template processing, and an alpha end-to-end run.
- 2027: migrate representative MML articles, build a native hammer baseline, and benchmark on top-level theorems, in the same units MizAR uses.
- 2028 and later: scale up the migration; learned premise selection; LLM-assisted failure recovery; MVM and extraction. Cryptographic and quantum targets stay future directions.

Read aloud:

- **2026: finish the specification, the Rust pipeline through the kernel, template processing, and an alpha end-to-end run.**
- **2027: migrate representative MML articles, build a native hammer baseline, and benchmark on top-level theorems, in the same units MizAR uses.**
- 2028 and later: scale up the migration; learned premise selection; LLM-assisted failure recovery; MVM and extraction. Cryptographic and quantum targets stay future directions.

5.2 - Back To The Two Paths

*The question is not whether higher-order systems
can use first-order ATPs. They clearly can.*

*The question is what becomes possible when automatic first-order
reasoning is a design principle from the beginning.*

Mizar Evo's answer, in six parts:

- ① Keep first-order logic and set theory as the base logic.
- ② Hide the first-order details with Mizar's language design.
- ③ Extend generic mathematics with templates.
- ④ Add checked computation with algorithms.
- ⑤ Rebuild fifty years of software infrastructure.
- ⑥ Combine LLMs and ATPs where each is strong.

Mizar Evolution: Why First-Order Logic, Now

└ Part 5. Roadmap And Closing

└ 5.2 - Back To The Two Paths

5.2 - Back To The Two Paths

*The question is not whether higher-order systems can use first-order ATPs. They clearly can.
The question is what becomes possible when automatic first-order reasoning is a design principle from the beginning.*

Mizar Evo's answer, in six parts:

- Keep first-order logic and set theory as the base logic.
- Hide the first-order details with Mizar's language design.
- Extend generic mathematics with templates.
- Add checked computation with algorithms.
- Rebuild fifty years of software infrastructure.
- Combine LLMs and ATPs where each is strong.

Read aloud:

- **The question is not whether higher-order systems can use first-order ATPs. They clearly can. The question is what becomes possible when automatic first-order reasoning is a design principle from the beginning.**
- Keep first-order logic and set theory as the base logic.
- Hide the first-order details with Mizar's language design.
- Extend generic mathematics with templates.
- Add checked computation with algorithms.
- Rebuild fifty years of software infrastructure.
- Combine LLMs and ATPs where each is strong.

5.3 - Closing

*Keep the foundation small.
Keep the mathematics readable.
Modernize everything else.*

- Thank you. I welcome your questions, especially on how to design the 2027 benchmark so that it compares like with like.

2026-10-07

Mizar Evolution: Why First-Order Logic, Now

└ Part 5. Roadmap And Closing

└ 5.3 - Closing

5.3 - Closing

*Keep the foundation small.
Keep the mathematics readable.
Modernize everything else.*

- Thank you. I welcome your questions, especially on how to design the 2027 benchmark so that it compares like with like.

Read aloud:

- **Keep the foundation small. Keep the mathematics readable. Modernize everything else.**
- **Thank you. I welcome your questions, especially on how to design the 2027 benchmark so that it compares like with like.**

Backup 1. MizAR 60 In Detail

Source: Jakubův, Chvalovský, Goertzel, Kaliszyk, Olšák, Piotrowski, Schulz, Suda, Urban, MizAR 60 for Mizar 50, ITP 2023.

- Dataset: MML 1147 exported by MPTP, 57,897 theorems including unnamed top-level lemmas; the same version as the MizAR 40 evaluation, so the results can be compared.
- 58.4% of top-level lemmas proved in the large-theory (hammering) mode, with no user help, by a portfolio limited to 420 s of CPU time (MizAR 40: about 40.6%).
- Over 75% proved when the premises can be chosen from the library by a human or a machine (MizAR 40: 56%).
- Strongest single method: 40% in 30 s in hammering mode; 60% in 120 s with human premises.
- Transfer: the strongest method also works on 13,370 new theorems from 242 new articles in MML 1382.
- Methods: E and Vampire with ENIGMA and Deepire guidance, learned premise selection, and a loop that trains on millions of ATP proofs.

Mizar Evolution: Why First-Order Logic, Now

└ Backup 1. MizAR 60 In Detail

└ Backup 1. MizAR 60 In Detail

Backup 1. MizAR 60 In Detail

Source: Jakubův, Chvalovský, Goertzel, Kaliszyk, Olšák, Piotrowski, Schulz, Suda, Urban, MizAR 60 for Mizar 50, ITP 2023.

- Dataset: MML 1147 exported by MPTP, 57,897 theorems including unnamed top-level lemmas; the same version as the MizAR 40 evaluation, so the results can be compared.
- 58.4% of top-level lemmas proved in the large-theory (hammering) mode, with no user help, by a portfolio limited to 420 s of CPU time (MizAR 40: about 40.6%).
- Over 75% proved when the premises can be chosen from the library by a human or a machine (MizAR 40: 56%).
- Strongest single method: 40% in 30 s in hammering mode; 60% in 120 s with human premises.
- Transfer: the strongest method also works on 13,370 new theorems from 242 new articles in MML 1382.
- Methods: E and Vampire with ENIGMA and Deepire guidance, learned premise selection, and a loop that trains on millions of ATP proofs.

Read aloud:

- Dataset: MML 1147 exported by MPTP, 57,897 theorems including unnamed top-level lemmas; the same version as the MizAR 40 evaluation, so the results can be compared.
- 58.4% of top-level lemmas proved in the large-theory (hammering) mode, with no user help, by a portfolio limited to 420 s of CPU time (MizAR 40: about 40.6%).
- Over 75% proved when the premises can be chosen from the library by a human or a machine (MizAR 40: 56%).
- Strongest single method: 40% in 30 s in hammering mode; 60% in 120 s with human premises.
- Transfer: the strongest method also works on 13,370 new theorems from 242 new articles in MML 1382.
- Methods: E and Vampire with ENIGMA and Deepire guidance, learned premise selection, and a loop that trains on millions of ATP proofs.

Optional notes and sources:

- Source: Jakubův, Chvalovský, Goertzel, Kaliszyk, Olšák, Piotrowski, Schulz, Suda, Urban, MizAR 60 for Mizar 50, ITP 2023.

Backup 2. Sledgehammer Evaluations In Detail

Prover	One-line / + Isar / + Oracle (% of 6,934 goals)
E	49.7 / 51.4 / 52.5
SPASS	49.4 / 50.5 / 52.0
Vampire	49.5 / 51.0 / 51.8
Z3	49.6 / 50.0 / 53.7

- Setup: 128 random AFP theories, up to 100 goals each; Isabelle2014; MePo filter; 30 s per prover in slices; replay must succeed within 2 s.
- Combined and trusted as oracles, the provers prove 60.7% of the goals.
- Judgement Day (2010): 1,240 subgoals from seven theories, 46% with E, SPASS, and Vampire in 30 s; a 2015 preliminary run reached 75% with six provers.
- Success per theory varies between 10% and 100%; Sledgehammer had been tuned on Judgement Day.

Mizar Evolution: Why First-Order Logic, Now

└ Backup 2. Sledgehammer Evaluations In Detail

└ Backup 2. Sledgehammer Evaluations In Detail

Backup 2. Sledgehammer Evaluations In Detail

Prover	One-line / + Isar / + Oracle (% of 6,934 goals)
E	49.7 / 51.4 / 52.5
SPASS	49.4 / 50.5 / 52.0
Vampire	49.5 / 51.0 / 51.8
Z3	49.6 / 50.0 / 53.7

- Setup: 128 random AFP theories, up to 100 goals each; Isabelle2014; MePo filter; 30 s per prover in slices; replay must succeed within 2 s.
- Combined and trusted as oracles, the provers prove 60.7% of the goals.
- Judgement Day (2010): 1,240 subgoals from seven theories, 46% with E, SPASS, and Vampire in 30 s; a 2015 preliminary run reached 75% with six provers.
- Success per theory varies between 10% and 100%; Sledgehammer had been tuned on Judgement Day.

Read aloud:

- Setup: 128 random AFP theories, up to 100 goals each; Isabelle2014; MePo filter; 30 s per prover in slices; replay must succeed within 2 s.
- Combined and trusted as oracles, the provers prove 60.7% of the goals.
- Judgement Day (2010): 1,240 subgoals from seven theories, 46% with E, SPASS, and Vampire in 30 s; a 2015 preliminary run reached 75% with six provers.
- Success per theory varies between 10% and 100%; Sledgehammer had been tuned on Judgement Day.

Optional notes and sources:

- Source: Blanchette, Haslbeck, Matichuk, Nipkow, Mining the Archive of Formal Proofs, CICM 2015, Section "Proof Automation with Sledgehammer", Figure 13; Böhme and Nipkow, IJCAR 2010.

Backup 3. HOL-To-FOL Encodings And Reconstruction

- Function application: a variable function applied to an argument becomes `app(F, X)`; applications of constants may stay curried or be flattened.
- Lambda abstraction: lambda lifting introduces fresh constants with defining equations; combinator translation is the alternative.
- Types: polymorphic HOL types are encoded by guards, tags, or monomorphization; the choice affects soundness, completeness, and prover performance.
- Booleans: Boolean-valued terms inside formulas need a separate encoding.
- Reconstruction: `metis` with the used lemmas, `smt` replay of SMT proofs, or generated `Isar` text; evaluations count reconstruction failures separately.

Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Mizar Evolution: Why First-Order Logic, Now

└ Backup 3. HOL-To-FOL Encodings And Reconstruction

└ Backup 3. HOL-To-FOL Encodings And Reconstruction

Backup 3. HOL-To-FOL Encodings And Reconstruction

- Function application: a variable function applied to an argument becomes `app(F, X)`; applications of constants may stay curried or be flattened.
- Lambda abstraction: lambda lifting introduces fresh constants with defining equations; combinator translation is the alternative.
- Types: polymorphic HOL types are encoded by guards, tags, or monomorphization; the choice affects soundness, completeness, and prover performance.
- Booleans: Boolean-valued terms inside formulas need a separate encoding.
- Reconstruction: `metis` with the used lemmas, `smt` replay of SMT proofs, or generated `Isar` text; evaluations count reconstruction failures separately.

Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Read aloud:

- Function application: a variable function applied to an argument becomes `app(F, X)`; applications of constants may stay curried or be flattened.
- Lambda abstraction: lambda lifting introduces fresh constants with defining equations; combinator translation is the alternative.
- Types: polymorphic HOL types are encoded by guards, tags, or monomorphization; the choice affects soundness, completeness, and prover performance.
- Booleans: Boolean-valued terms inside formulas need a separate encoding.
- Reconstruction: `metis` with the used lemmas, `smt` replay of SMT proofs, or generated `Isar` text; evaluations count reconstruction failures separately.

Optional notes and sources:

- Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Backup 4. Template Instantiation: One Proof, Many Views

Bounded type parameter and generic theorem (specification example, proof omitted):

```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

Instantiation **specification example** (with the required registrations)

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma] :: additive view
```

- A ring reaches Magma along two paths; qua selects the view, and the view also sets the notation.

Mizar Evolution: Why First-Order Logic, Now

└ Backup 4. Template Instantiation: One Proof, Many Views

└ Backup 4. Template Instantiation: One Proof, Many Views

Backup 4. Template Instantiation: One Proof, Many Views

Bounded type parameter and generic theorem (specification example, proof omitted):

```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T
      p being Permutation of dom s
      holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

Instantiation **specification example** (with the required registrations)

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma] :: additive view
```

- A ring reaches Magma along two paths; qua selects the view, and the view also sets the notation.

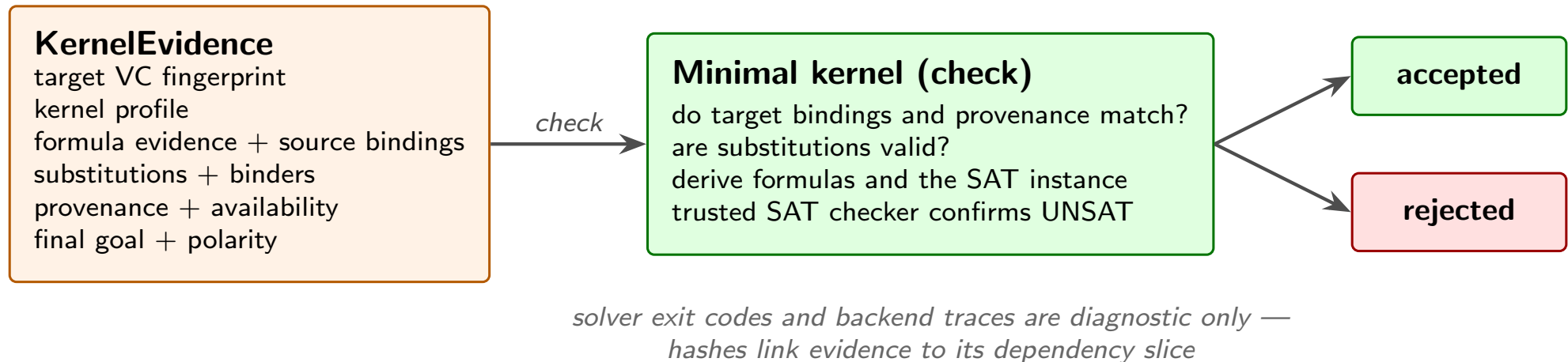
Read aloud:

- Bounded type parameter and generic theorem.
- A ring reaches Magma along two paths; qua selects the view, and the view also sets the notation.

Optional notes and sources:

- Source: doc/spec/en/18.templates.md, section 18.2.2.

Backup 5. Kernel Evidence And The Trusted SAT Check



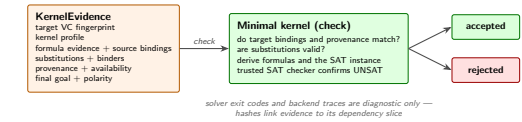
- The evidence holds source formulas, explicit substitutions, provenance, and target and goal bindings. The kernel checks them, builds a deterministic SAT problem, and requires UNSAT from its trusted in-process SAT checker.
- Backend proof traces, SMT proof objects, logs, and exit codes are for diagnosis only.

Mizar Evolution: Why First-Order Logic, Now

└ Backup 5. Kernel Evidence And The Trusted SAT Check

└ Backup 5. Kernel Evidence And The Trusted SAT Check

Backup 5. Kernel Evidence And The Trusted SAT Check



- The evidence holds source formulas, explicit substitutions, provenance, and target and goal bindings. The kernel checks them, builds a deterministic SAT problem, and requires UNSAT from its trusted in-process SAT checker.
- Backend proof traces, SMT proof objects, logs, and exit codes are for diagnosis only.

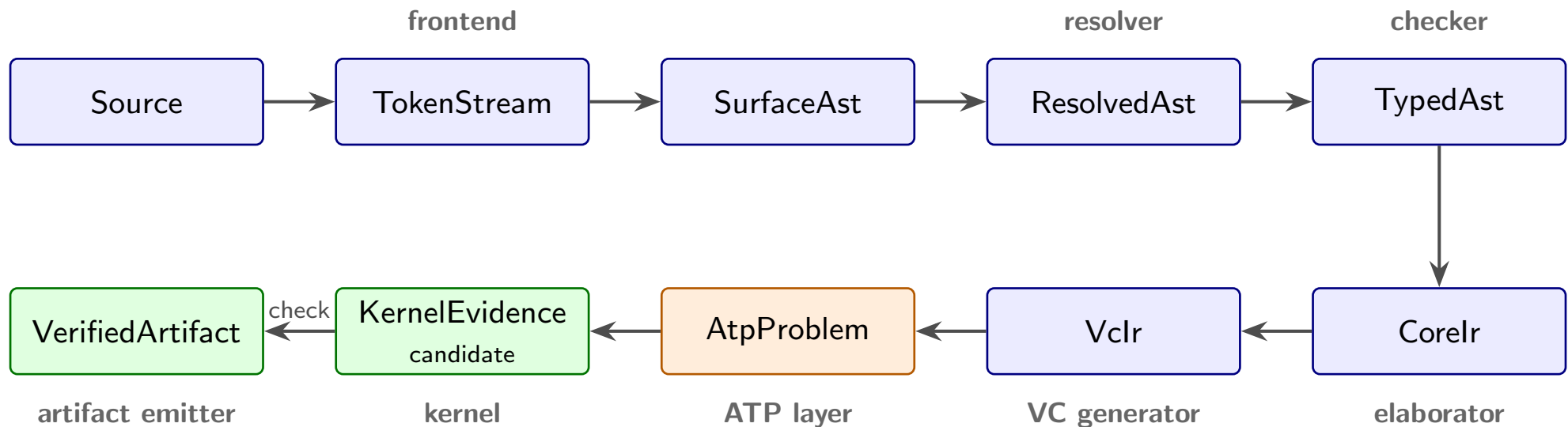
Read aloud:

- The evidence holds source formulas, explicit substitutions, provenance, and target and goal bindings. The kernel checks them, builds a deterministic SAT problem, and requires UNSAT from its trusted in-process SAT checker.
- Backend proof traces, SMT proof objects, logs, and exit codes are for diagnosis only.

Optional notes and sources:

- Source: `doc/design/architecture/en/08.reasoning_boundary.md`,
`15.kernel_certificate_format.md`.

Backup 6. The Core ATP Path



- Only obligations that are still open after deterministic discharge go to ATPs. The earlier discharge also needs replayable evidence.
- Each boundary states who owns a fact, which artifact records it, and what must be checked again after a change.

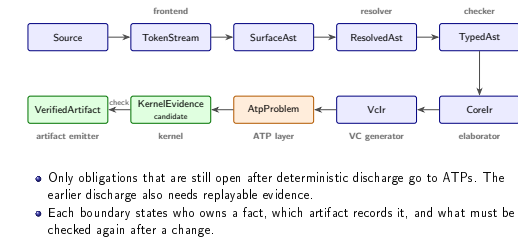
2026-10-07

Mizar Evolution: Why First-Order Logic, Now

└ Backup 6. The Core ATP Path

└ Backup 6. The Core ATP Path

Backup 6. The Core ATP Path



Read aloud:

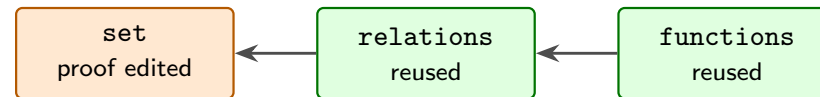
- Only obligations that are still open after deterministic discharge go to ATPs. The earlier discharge also needs replayable evidence.
- Each boundary states who owns a fact, which artifact records it, and what must be checked again after a change.

Optional notes and sources:

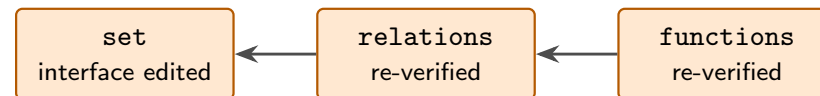
- Source: `doc/design/architecture/en/00.pipeline_overview.md`; Bialystok deck, Part 10.

Backup 7. Incremental Verification

theorem proof edit:
public statement + accepted status unchanged — importers reuse



interface change:
the dependency cone re-verifies



reuse requires all relevant keys: source/dependency slices, package/lockfile, toolchain/schema, registration/cluster, verifier policy, evidence/witnesses; missing data is a cache miss — a clean build must reproduce every acceptance

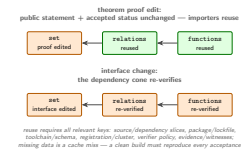
- A proof-body edit does not rebuild importers when the exported statement and the accepted status stay the same. An interface change re-verifies the dependency cone.
- All relevant cache keys must match; missing data is a cache miss. Cache reuse is never proof authority: a clean build must reproduce every acceptance.

Mizar Evolution: Why First-Order Logic, Now

└ Backup 7. Incremental Verification

└ Backup 7. Incremental Verification

Backup 7. Incremental Verification



- A proof-body edit does not rebuild importers when the exported statement and the accepted status stay the same. An interface change re-verifies the dependency cone.
- All relevant cache keys must match; missing data is a cache miss. Cache reuse is never proof authority: a clean build must reproduce every acceptance.

Read aloud:

- A proof-body edit does not rebuild importers when the exported statement and the accepted status stay the same. An interface change re-verifies the dependency cone.
- All relevant cache keys must match; missing data is a cache miss. Cache reuse is never proof authority: a clean build must reproduce every acceptance.

Optional notes and sources:

- Source: `doc/design/architecture/en/11.artifact_and_incremental_build.md`, `18.dependency_fingerprint.md`; Bialystok deck, Story 5.

Backup 8. Package Manifest

Mizar Evo **specification example**

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

- Reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence.
- Versioned reuse replaces manual copying between article sets; an aggregate module can export a whole topic through one import.

Source: doc/spec/en/23.package_management_and_build_system.md; Bialystok deck, Story 1.

Mizar Evolution: Why First-Order Logic, Now

└ Backup 8. Package Manifest

└ Backup 8. Package Manifest

Backup 8. Package Manifest

Mizar Evo **specification example**

```
[package]
name = "algebra"
version = "2.3.1"
edition = "2025"

[dependencies]
mml_core = "1.0"
topology = { version = "0.9", features = ["metric"] }
```

- Reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence.
- Versioned reuse replaces manual copying between article sets; an aggregate module can export a whole topic through one import.

Source: doc/spec/en/23.package_management_and_build_system.md; Bialystok deck, Story 1.

Read aloud:

- Reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence.
- Versioned reuse replaces manual copying between article sets; an aggregate module can export a whole topic through one import.

Optional notes and sources:

- Source: doc/spec/en/23.package_management_and_build_system.md; Bialystok deck, Story 1.

Backup 9. Forty-Five-Minute Additions

The story does not change. Add depth with these examples, in this order of priority:

- ① HOL-to-FOL function encoding worked through on one goal (app, partial application, lambda lifting).
- ② Mizar function representation: `Function of X,Y, f.x`, soft type as a set-theoretic object.
- ③ Template: `PermProduct [T]` with additive and multiplicative views (Backup 4).
- ④ Algorithm: Euclidean GCD plus `by computation` and the promotion to a functor.
- ⑤ Structures and views: `AddMagma`, `MulMagma`, `Magma` (Bialystok Story 2).
- ⑥ Modern infrastructure: `mizar.pkg`, namespaces, the fingerprint graph (Backups 7-8).
- ⑦ Trusted boundary: ATP search, `KernelEvidence`, checked acceptance (Backup 5).

Mizar Evolution: Why First-Order Logic, Now

└ Backup 9. Forty-Five-Minute Additions

└ Backup 9. Forty-Five-Minute Additions

Backup 9. Forty-Five-Minute Additions

The story does not change. Add depth with these examples, in this order of priority:

- HOL-to-FOL function encoding worked through on one goal (app, partial application, lambda lifting).
- Mizar function representation: `Function of X,Y, f.x`, soft type as a set-theoretic object.
- Template: `PermProduct [T]` with additive and multiplicative views (Backup 4).
- Algorithm: Euclidean GCD plus by `computation` and the promotion to a functor.
- Structures and views: `AddMagma`, `MulMagma`, `Magma` (Bialystok Story 2).
- Modern infrastructure: `mizar.pkg`, namespaces, the fingerprint graph (Backups 7-8).
- Trusted boundary: ATP search, `KernelEvidence`, checked acceptance (Backup 5).

Read aloud:

- The story does not change. Add depth with these examples, in this order of priority:
- HOL-to-FOL function encoding worked through on one goal (app, partial application, lambda lifting).
- Mizar function representation: `Function of X,Y, f.x`, soft type as a set-theoretic object.
- Template: `PermProduct [T]` with additive and multiplicative views (Backup 4).
- Algorithm: Euclidean GCD plus by `computation` and the promotion to a functor.
- Structures and views: `AddMagma`, `MulMagma`, `Magma` (Bialystok Story 2).
- Modern infrastructure: `mizar.pkg`, namespaces, the fingerprint graph (Backups 7-8).
- Trusted boundary: ATP search, `KernelEvidence`, checked acceptance (Backup 5).

Backup 10. Sources And Attribution

Exact MML excerpts used in this talk:

Purpose	Source	Lines	Used in
Function application f.x	funct_1.miz	138-140	Frame 2.3
Function of X,Y as a soft type	funct_2.miz	87-90	Frame 2.3

Source URLs:

- FUNCT_1: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- FUNCT_2: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz

Attribution note:

- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.
- Benchmark figures: see Backups 1-2 and references.bib. Verify bibliographic metadata against publishers before the final deck.

Mizar Evolution: Why First-Order Logic, Now

└ Backup 10. Sources And Attribution

└ Backup 10. Sources And Attribution

Backup 10. Sources And Attribution

Exact MML excerpts used in this talk:

Purpose	Source	Lines	Used in
Function application $f.x$	<code>funct_1.miz</code>	138-140	Frame 2.3
Function of X,Y as a soft type	<code>funct_2.miz</code>	87-90	Frame 2.3

Source URLs:

- FUNCT_1: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- FUNCT_2: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz

Attribution note:

- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.
- Benchmark figures: see Backups 1-2 and `references.bib`. Verify bibliographic metadata against publishers before the final deck.

Read aloud:

- FUNCT_1: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- FUNCT_2: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz
- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.
- Benchmark figures: see Backups 1-2 and `references.bib`. Verify bibliographic metadata against publishers before the final deck.